

Everything a VLA bolts onto a transformer

Same self-attention block throughout. What changes between an LLM, a VLM and a VLA is only what gets spliced into the input sequence and what gets spliced into the output vocabulary — plus, for a VLA, a loop back out into the world.

■ shared / unchanged ■ added by VLM ■ added by VLA

Figure 1 — the same core, two things spliced in

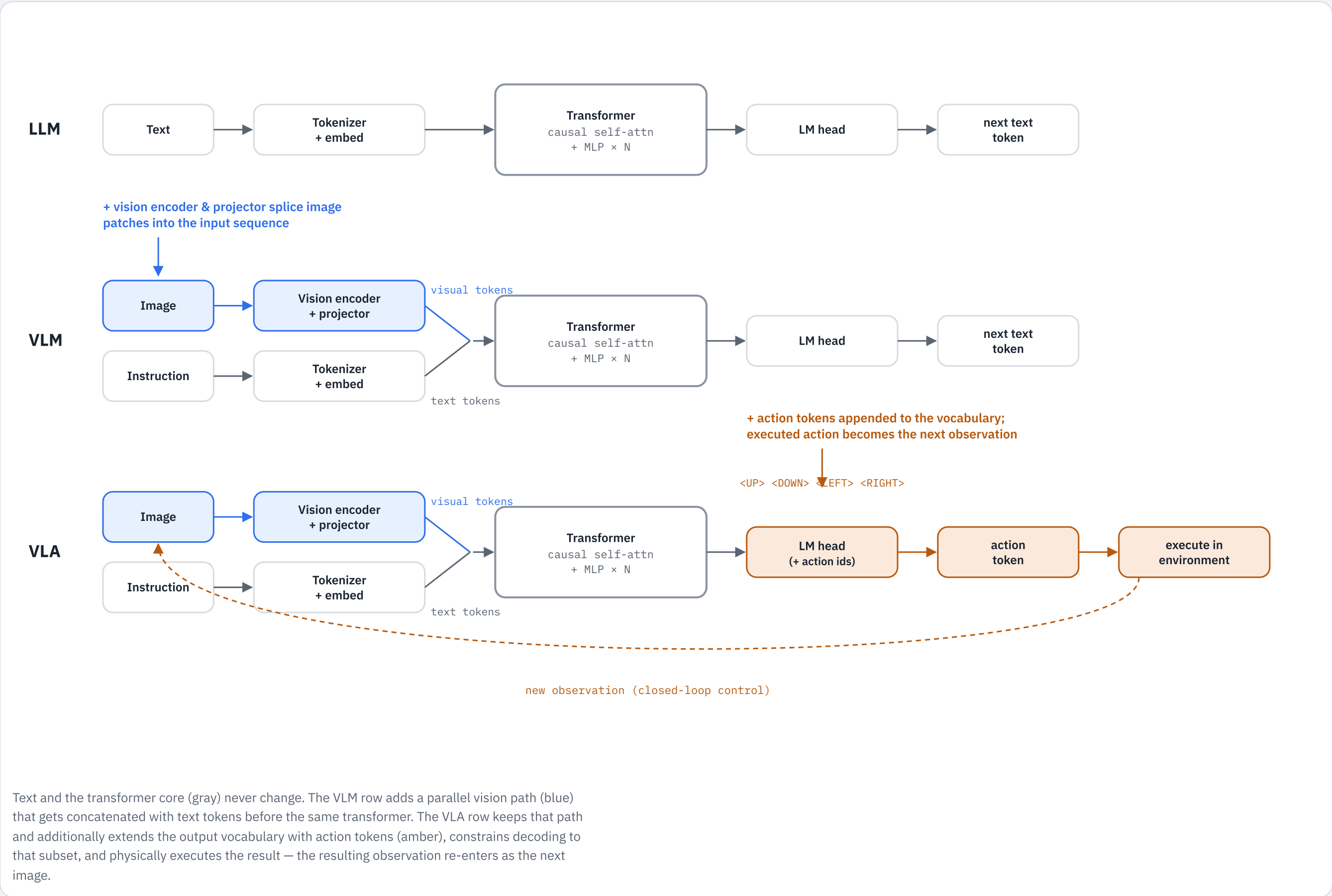
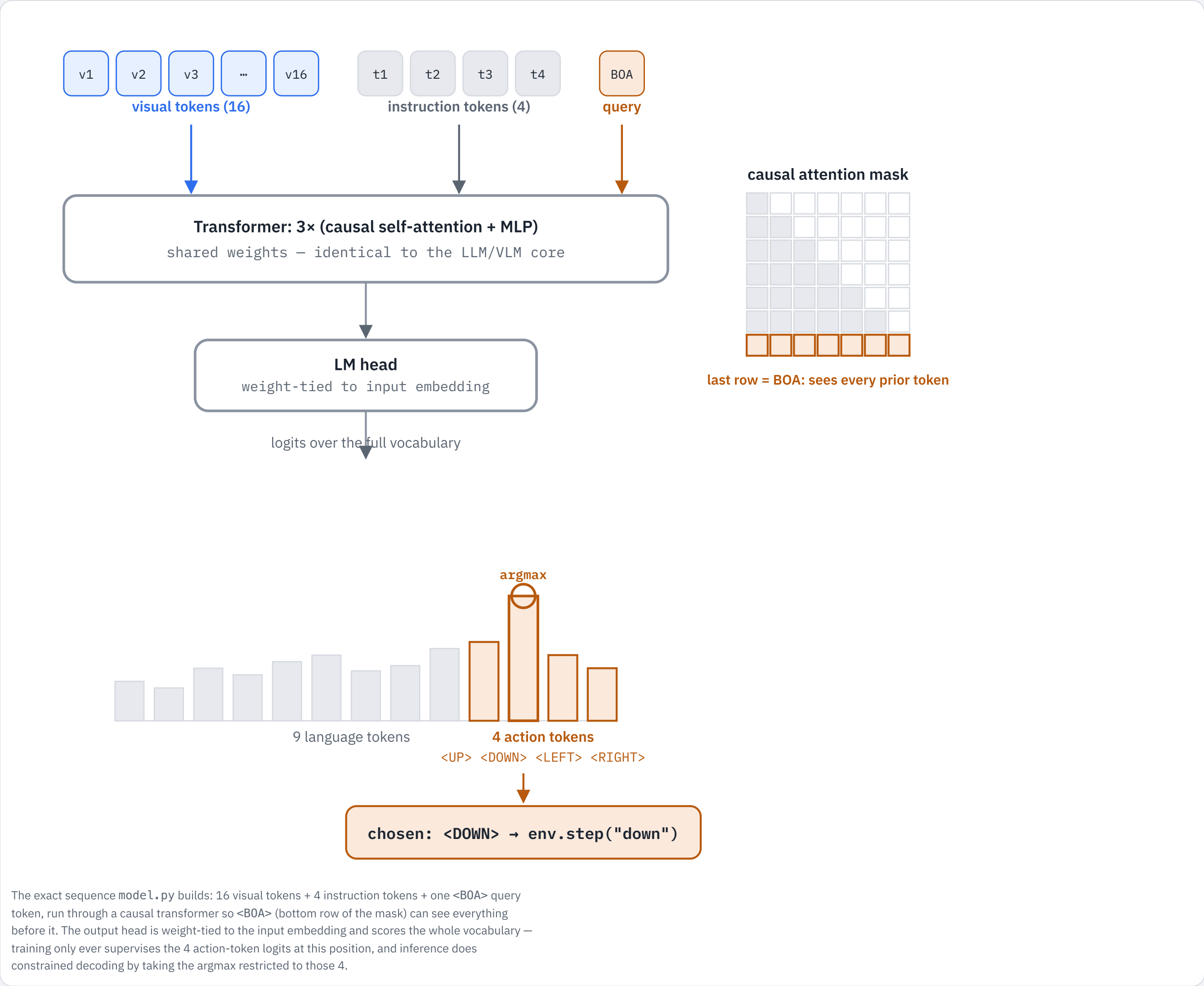


Figure 2 — inside the VLA's single forward pass



Same ingredients, three recipes

	LLM	VLM	VLA
Input tokens	text only	text + image patches (projected)	text + image patches (projected)
Vocabulary	language tokens	language tokens	language tokens + action tokens
Output head	softmax over vocab	softmax over vocab	softmax over vocab, decoding constrained to action ids
Training loss	next-token cross-entropy	next-token cross-entropy (captions / VQA)	cross-entropy vs. the expert's action token (behavior cloning)
Control loop	none — batch generation	none — one question, one answer	closed loop: action → <code>env.step()</code> → new image → repeat
In your <code>vla_scratch/</code>	—	<code>model.py</code> : VisionEncoder + projector	+ <code>tokenizer.py</code> action ids, + <code>env.py</code> render/step loop

Reference: RT-2 and OpenVLA overload a real pretrained LLM's vocabulary this same way; your toy model does it explicitly with four hand-picked `<UP>/<DOWN>/<LEFT>/<RIGHT>` tokens so every step of the trick stays visible.